

## 1 Introduction

Suite à vos retours sur les sections précédentes, Tali'Zorah Vas Normandy (spécialiste en développement et en intelligence artificielle) vient de nous informer que nous touchons au but. Nous disposons maintenant de toutes les pièces nécessaires à la construction du protocole de chiffrement Geth. Il est temps de s'y attaquer !

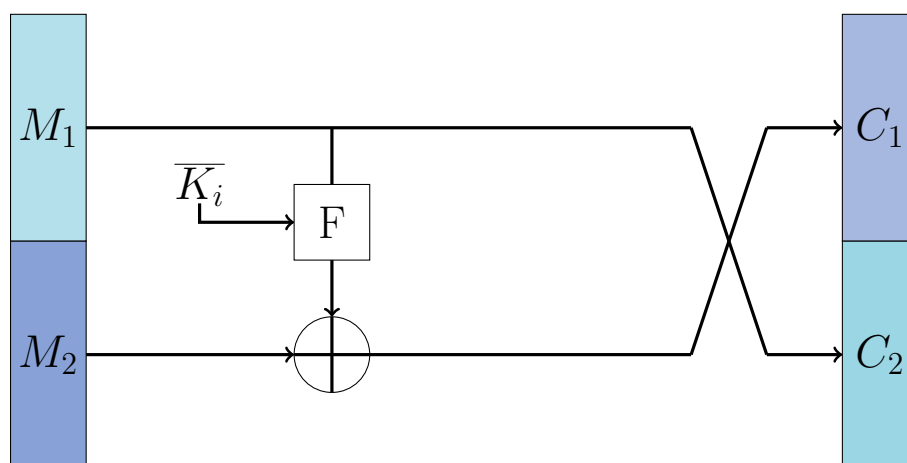
## 2 Principe de l'algorithme

D'après la documentation du module de challenge, le protocole de chiffrement Geth fonctionne suivant un schéma de Feistel.

### 2.1 Caractéristiques

Le schéma de Feistel utilisé travaille sur des messages binaires  $M$  de 64 bits (manipulés comme des chaînes de 64 caractères) et utilise une clé unique  $K$  de 8 caractères ASCII qui permettra de générer 2 clés partielles  $\overline{K}_1$  et  $\overline{K}_2$ .

### 2.2 Schéma du tour $i$

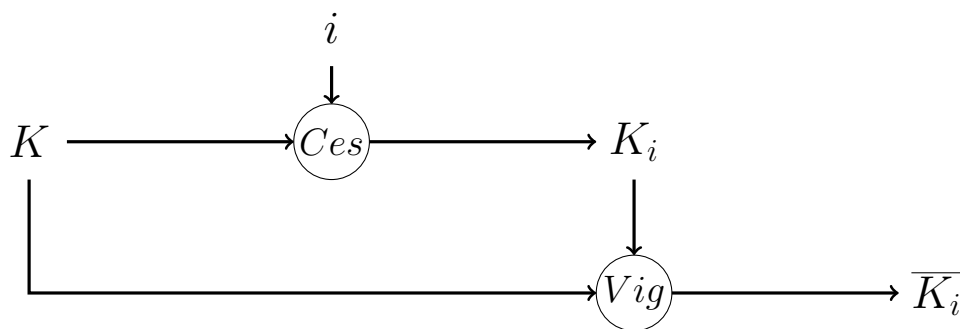


Dans ce schéma,  $M_1$  représente les 32 premiers caractères de  $M$  et  $M_2$  les 32 derniers caractères,  $\overline{K}_i$  représente la  $i$ -ème clé partielle générée à partir de  $K$  (voir partie 2.3) et  $F$  est une fonction non linéaire présentée dans la partie 2.4.

Comme indiqué précédemment, le schéma complet est composé de deux tours ( $i = 1$  et  $i = 2$ ) ayant le même schéma (seule la clé partielle utilisée change entre les deux tours).

### 2.3 Génération des deux clés partielles

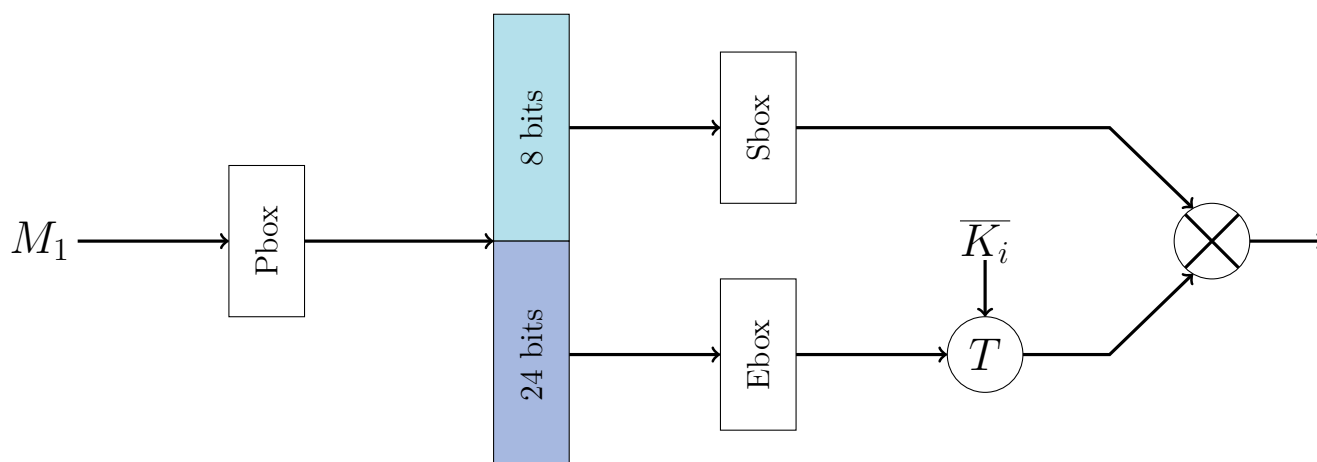
Le schéma utilise une clé unique : une chaîne de 8 caractères. Celle-ci est utilisée pour générer deux clés partielles  $\overline{K}_1$  et  $\overline{K}_2$  qui servent respectivement au tour 1 et au tour 2. Le procédé de génération de  $\overline{K}_i$  à partir de la clé  $K$  est le suivant :



Dans ce schéma, *Ces* et *Vig* représentent respectivement les algorithmes de César et de Vigenère.

## 2.4 La fonction $F$

La fonction non linéaire utilisée dans notre schéma est la suivante :



Dans ce schéma :

- la *Pbox* utilisée est celle décrite dans le CM (slide 27)
- la *Sbox* utilisée est celle décrite par le fichier joint au projet
- la *Ebox* utilisée consiste à répéter un bit sur trois (ainsi le message abcdef deviendra aabcddef).
- $T$  représente l'algorithme de transposition (dont la clé sera  $\overline{K_i}$  et le message sera ce qui sortira de la *Ebox*)
- la dernière opération est une addition modulo  $2^{32}$

## 2.5 Structure principale

Pour réaliser cet algorithme, il vous est conseillé d'utiliser la structure suivante :

| AlgorithmeFeistel                                                            |
|------------------------------------------------------------------------------|
| -sbox : string[]                                                             |
| +AlgorithmeFeistel()                                                         |
| +Pbox(message : string) : string                                             |
| +Sbox(message : string) : string                                             |
| +Ebox(message : string) : string                                             |
| +Add(nb1 : string, nb2 : string) : string                                    |
| +CreationCle(cle : string, numTour : int) : string                           |
| +F(message : string, cle : string) : string                                  |
| +TourDeChiffrement(message : string, cle : string, numTour : int) : string   |
| +TourDeDechiffrement(message : string, cle : string, numTour : int) : string |

**Le constructeur** initialisera l'attribut sbox en lisant le fichier Sbox.txt et en convertissant chacune des chaînes hexadécimales ("0x...") en binaire à l'aide d'une méthode codée lors de la section Opérations Binaires.

**PBox, SBox et EBox** représentent les trois boxes utilisées dans la fonction non linéaire  $F$ .

**Add** doit réaliser l'addition modulo  $2^{32}$  des deux nombres donnés en entrée. Attention, les entrées et la sortie sont en binaire et représentent des entiers non signés. On utilisera de nouveau la section Opérations Binaires.

**F** représente la fonction non linéaire présentée dans la partie précédente.

**CreationCle** sert à créer une clé partielle à partir de la clé d'origine et du numéro du tour.

**TourDeChiffrement** réalise un seul tour de l'algorithme de chiffrement.

**TourDeDechiffrement** réalise un seul tour de l'algorithme de déchiffrement.

## 2.6 Challenges

### Challenge FTL1 : Permutation Box :

Le serveur enverra un nombre aléatoire de tests et terminera la discussion par "FIN". Pour chaque test le serveur :

- envoie d'un message binaire (32 caractères 0 ou 1).
- attend l'image de ce message par la PBox (représentation binaire sur 32 bis).
- validera ou non votre réponse par l'envoi de OK ou NOK|...

6. Codez la ou les méthodes utiles de la classe *AlgorithmeFeistel*.

7. Dans le répertoire *Challenges/Realisations*, créez et implémentez le challenge.

8. Ajoutez ce nouveau challenge à votre fabrique.

9. Vérifiez que vous validez ce challenge.

### Challenge FTL2 : Substitution Box :

Le serveur enverra un nombre aléatoire de tests et terminera la discussion par "FIN". Pour chaque test le serveur :

- envoie du message sous la forme d'une représentation binaire sur 8 bits d'un entier non signé.
- attend l'image de ce message par la SBox (représentation binaire sur 32 bis).
- validera ou non votre réponse par l'envoi de OK ou NOK|...

10. Codez la ou les méthodes utiles de la classe *AlgorithmeFeistel*.

11. Dans le répertoire *Challenges/Realisations*, créez et implémentez le challenge.

12. Ajoutez ce nouveau challenge à votre fabrique.

13. Vérifiez que vous validez ce challenge.

### Challenge FTL3 : Expansion Box :

Le serveur enverra un nombre aléatoire de tests et terminera la discussion par "FIN". Pour chaque test le serveur :

- envoie d'un message binaire (24 caractères 0 ou 1).
- attend l'image de ce message par la EBox (représentation binaire sur 32 bis).
- validera ou non votre réponse par l'envoi de OK ou NOK|...

14. Codez la ou les méthodes utiles de la classe *AlgorithmeFeistel*.

15. Dans le répertoire *Challenges/Realisations*, créez et implémentez le challenge.

16. Ajoutez ce nouveau challenge à votre fabrique.

17. Vérifiez que vous validez ce challenge.

### Challenge FTL4 : Création des clés :

Le serveur enverra un nombre aléatoire de tests et terminera la discussion par "FIN". Pour chaque test le serveur :

- envoie de la clé (string de 8 caractères).
- envoie du numéro du tour.
- attend la clé partielle générée à partir de la clé et du numéro de tour.
- validera ou non votre réponse par l'envoi de OK ou NOK|...

14. Codez la ou les méthodes utiles de la classe *AlgorithmeFeistel*.

15. Dans le répertoire *Challenges/Realisations*, créez et implémentez le challenge.

16. Ajoutez ce nouveau challenge à votre fabrique.

17. Vérifiez que vous validez ce challenge.

### Challenge FTL5 : Fonction sans retour :

Le serveur enverra un nombre aléatoire de tests et terminera la discussion par "FIN". Pour chaque test le serveur :

- envoie d'un message binaire (32 caractères 0 ou 1).
- envoie de la clé (string de 8 caractères).
- attend le résultat de la fonction F.
- validera ou non votre réponse par l'envoi de OK ou NOK|...

14. Codez la ou les méthodes utiles de la classe *AlgorithmeFeistel*.

15. Dans le répertoire *Challenges/Realisations*, créez et implémentez le challenge.

16. Ajoutez ce nouveau challenge à votre fabrique.

17. Vérifiez que vous validez ce challenge.

### Challenge FTL6 : Chiffrement :

Le serveur enverra un nombre aléatoire de tests et terminera la discussion par "FIN". Pour chaque test le serveur :

- envoie d'un message binaire (64 caractères 0 ou 1).
- envoie de la clé (string de 8 caractères).
- attend le chiffrement de ce message par le protocole de chiffrement Geth à l'aide de la clé fournie.
- validera ou non votre réponse par l'envoi de OK ou NOK|...

14. Codez la ou les méthodes utiles de la classe *AlgorithmeFeistel*.

15. Dans le répertoire *Challenges/Realisations*, créez et implémentez le challenge.

16. Ajoutez ce nouveau challenge à votre fabrique.

17. Vérifiez que vous validez ce challenge.

### Challenge FTL7 : Déchiffrement :

Le serveur enverra un nombre aléatoire de tests et terminera la discussion par "FIN". Pour chaque test le serveur :

- envoie d'un message chiffré binaire (64 caractères 0 ou 1).
- envoie de la clé (string de 8 caractères).
- attend le déchiffrement de ce message par le protocole de chiffrement Geth à l'aide de la clé fournie.
- validera ou non votre réponse par l'envoi de OK ou NOK|...

14. Codez la ou les méthodes utiles de la classe *AlgorithmeFeistel*.

15. Dans le répertoire *Challenges/Realisations*, créez et implémentez le challenge.

16. Ajoutez ce nouveau challenge à votre fabrique.

17. Vérifiez que vous validez ce challenge.

### **Challenge FTL8 : Le dernier challenge :**

L'unité Geth Légion a constaté l'existence d'un huitième challenge non documenté dans cette section. Après rétro-ingénierie, Tali'Zorah vas Normandy semble penser qu'il s'agit d'un artefact de sécurité Geth dont le déblocage ne peut être réalisé via le module de challenge, mais directement en s'attaquant au système de sauvegarde du module. Deux outils de lecture/écriture de fichiers en binaire ont été mis à votre disposition. Bon courage !